

Day 5 Lab Sheet: Put the Notes Board on the Internet

Goal: By the end of this lab your Notes Board code has three small final changes, it is pushed to GitHub, and (if Render accepts your account) it is live on a public link. You will also see what happens to the data on the Free plan.

Time: about 100 minutes, plus the quiz.

What you need: Your `notes-board` folder from Day 4. It is your GitHub repository. Your GitHub account. A working `.env` file in the `notes-board` folder.

If your Day 4 work is missing or broken, copy the **Day 5 starter** folder from your instructor. It has everything finished up to Day 4. The **done** folder is the finished result for today.

About this sheet and Render. The Render steps were written from Render's documentation (read on 6 October 2026), not from a live test of the dashboard. Menu names and button labels may be different on your screen. Wherever you see **(check against your dashboard, not verified)**, read your screen and use the nearest name. Ask your instructor if you are unsure.

Success tiers: Tier 1 means it works on localhost (this is a pass). Tier 2 means it is pushed to GitHub. Tier 3 means it is live on Render.

Part 0: Check the starter and run it on your computer (5 minutes)

1. Open VS Code. Open your `notes-board` folder (**File, Open Folder**).
2. Open a terminal. Choose **Command Prompt** (the small down arrow next to the **+** icon).
3. Check that the terminal is inside the `notes-board` folder. The prompt must end with `notes-board>`.
4. Check the tools:

```
node -v
```

You should see a version that starts with `v24`.

5. Check that `package.json` is in this folder:

```
dir
```

Check: You see `package.json`, `server.js`, `data.js`, a `public` folder and a `.env` file. If you do not see `.env`, run `dir /a` to list all files, including hidden ones. If it is still missing, copy `.env.example` to `.env` and write a password in it (you did this on Day 3 or Day 4).

6. Install the packages (it is safe to run this again) and start the app:

```
npm install
```

```
npm start
```

7. Open `http://localhost:3000` in the browser.

Check: You see the Notes Board with notes. Adding a note with your password works. This is **Tier 1**.

8. In the terminal press **Ctrl + C** to stop the server.

If the server stops with `NOTES_PASSWORD is not set.`, see the Stuck Sheet.

Part 1: Three small code changes (15 minutes)

We prepare the app for hosting.

1a. Add the `/health` route

1. Open `server.js`.
2. Find the line that says `// API 1: read all notes (public, no password)`.
3. Click at the **start** of that line, and paste or type this block **above** it, with one empty line before and after:

File: `server.js`

```
// A tiny page that says "I am alive". Hosting services can use it to check the app.
app.get("/health", function (request, response) {
  response.json({ ok: true });
});
```

1b. Listen on `0.0.0.0` and on the `PORT` number

1. Scroll to the **end** of `server.js`. Find the block that starts with `// Prepare the database first, then start listening.`
2. **Replace the whole block** (from that comment to the last `});`) with this:

File: server.js

```
// Prepare the database first, then start listening.
init().then(function () {
  // "0.0.0.0" means: accept visitors from the internet, not only this computer.
  // A hosting service tells us the port through the PORT setting.
  app.listen(PORT, "0.0.0.0", function (error) {
    if (error) {
      console.error("Could not start the server: " + error.message);
      console.error("Is another server already using port " + PORT + "?");
      process.exit(1);
    }
    console.log("Notes Board is running on port " + PORT);
    console.log("On your own computer, open http://localhost:" + PORT);
  });
});
```

3. Look at the top of `server.js`. This line must already be there. If it is not, add it under the line that reads `const PASSWORD = process.env.NOTES_PASSWORD;`:

File: server.js

```
const PORT = process.env.PORT || 3000;
```

4. Save the file (**Ctrl + S**).

1c. Check package.json

Open `package.json`. It must have the `start` script and the `engines` part, as below. Your file may have a different `description`. That is fine. If a part is missing, add it. Mind the commas.

File: package.json

```
{
  "name": "notes-board",
  "version": "1.0.0",
  "description": "A small Notes Board built in the Web Technology workshop at Mandsaur University.",
  "main": "server.js",
  "private": true,
  "scripts": {
    "start": "node server.js"
  },
  "engines": {
    "node": ">=24 <25"
  },
  "dependencies": {
    "express": "^5.2.1"
  }
}
```

Do not change the version number of `express` in your own file if it is different. Keep what `npm install` wrote.

1d. Create `README.md`

1. In VS Code, click the **New File** icon in the explorer, and name the file `README.md` (in the `notes-board` folder, not inside `public`).
2. Type this. You may write your own name in the first line.

File: `README.md`

Notes Board

A small full-stack web app built in the Web Technology workshop at Mandsaur University.

- Frontend: HTML, CSS and JavaScript (in `public/`)
- Backend: Node.js and Express (`server.js`)
- Database: SQLite (`data.js`)

Run it on your computer

1. Install Node.js 24 LTS.
2. Run `npm install`.
3. Copy `.env.example` to `.env` and write your own password in it.
4. Run `npm start` and open `http://localhost:3000`.

Notes

- Anyone can read the notes. You need the password to add or delete a note.
- The `.env` file is private and is not part of this repository.

3. Save the file.

1e. Test the changes

1. Run `npm start` in the terminal.
2. Open `http://localhost:3000/health`.

Check: The page shows `{"ok":true}`. The terminal shows `Notes Board is running on port 3000`.

3. Open `http://localhost:3000`. The Notes Board still works.
4. Press **Ctrl + C** in the terminal to stop the server.

Part 2: Commit and push (10 minutes)

This is the same workflow as Day 4. We do it again to prove that you know it.

1. Check what changed:

```
git status
```

Check: You see `server.js` as modified and `README.md` as a new file. You must **not** see `.env` in the list. If you see `.env`, stop and call your instructor.

2. Add, commit and push:

```
git add .
```

```
git commit -m "Add health route, README and hosting settings"
```

```
git push
```

3. If a sign-in window opens, sign in to GitHub with your account.

4. Open your repository on `github.com` in the browser and refresh the page.

Check:

- You see the `README.md` text under the file list.
- You see the newest commit message.
- You do **not** see a `.env` file in the list. (`.env.example` is fine. It has no real password.)

This is **Tier 2**.

If `git push` fails, see the Stuck Sheet.

Part 3: Create a Render account (10 minutes)

1. Open the Render website in the browser. Search for `Render cloud hosting` or ask your instructor for the address.
2. Look for **Get Started, Sign up** or a similar button.
3. Choose **GitHub** to sign up (sign in with GitHub). (**check against your dashboard, not verified**)
4. GitHub may ask you to **authorize** Render. Read the screen. Allow only what is needed for Render to read your repositories.
5. If Render asks you for more details, fill them in. If Render asks you for a **card** or a **phone number**, **stop and tell your instructor**. Whether Render asks for them for the Free plan is **not verified**. Do not enter a card.
6. When you see the Render dashboard, this part is done.

Check: You are on the Render dashboard.

If sign-up does not work for you: This can happen. Many students sign up from one network, and Render may block it. We do not know in advance. You stay at **Tier 2**, which is a good result. Watch the projector, where your instructor deploys a shared demo, and follow along on paper. Do the extras in Part 10.

Part 4: Create the web service (10 minutes)

1. On the dashboard, click **New**, then **Web Service**. (**check against your dashboard, not verified**)
2. Render asks where the code is. You may see these options: **Git Provider**, **Public Git Repository** and **Existing Image**.
3. Choose **Git Provider**. Do **not** choose Public Git Repository. Render's documentation says that auto-deploys work with the Git Provider route and not with the public repository route.
4. If you do not see your repository, click the link to configure or connect GitHub, and allow Render to see your `notes-board` repository. Then choose it.
5. Fill in the form. If a field is not listed here, leave it as it is.

Field	What to write
Name	<code>yourname-notes-board</code> (small letters, no spaces)
Region	The region nearest to India that you see in the list, or the default
Branch	<code>main</code> (check the branch you pushed. Run <code>git branch</code> in the terminal if you are not sure)
Language	<code>Node</code>
Build Command	<code>npm install</code>
Start Command	<code>npm start</code>
Instance type or plan	Free

6. If you see a **Root Directory** field, leave it empty. Your `package.json` is at the top of your repository.
7. **Do not click Create Web Service yet.** First do Part 5.

Check: The Free plan is selected, and Build Command and Start Command are filled in.

Part 5: Set the environment variable (5 minutes)

Your password must be set in Render. It must **never** be in your code or in GitHub.

1. On the same form, find **Advanced** and open it. Look for **Environment Variables** or **Add Environment Variable. (check against your dashboard, not verified)** If you cannot find it, you can add it after the service is created (see "If you forgot" below).
2. Add one variable:

Key	Value
NOTES_PASSWORD	A new password that you choose. Not your email password. Not your GitHub password. Write it on paper

3. The key must be written exactly: `NOTES_PASSWORD` , capital letters and an underscore.
4. Do **not** add a `PORT` variable. Render gives the port to your app by itself.
5. Now click **Create Web Service**.

If you forgot: Render will build, but the app will stop with `NOTES_PASSWORD is not set.` in the log. Open your service, click **Environment**, click **Add Environment Variable**, add the key and the value, and save. Render may start a new deploy by itself. If not, use **Manual Deploy. (check against your dashboard, not verified)**

Part 6: First deploy and read the log (15 minutes)

1. After you click **Create Web Service**, Render opens your service page. Click **Deploys** or **Logs** (the name on your screen may be different). **(check against your dashboard, not verified)**
2. Watch the log lines appear. You are looking for these things:
 - Lines where the code is taken from GitHub.
 - Lines from `npm install` . They show packages being added.
 - A line that shows which Node.js version is used. Is it version 24? Write it here:

 - Our own lines: `No .env file found. Using the settings of this computer.` and then `Notes Board is running on port ...` (the port number is chosen by Render).
3. When the deploy is finished, the service page shows a status such as **Live**. The wording is **not verified**.

How long a first deploy takes is **not verified**. Wait for it, and keep reading the log. Do not click Create again.

- The message `No .env file found.` is **normal** on Render. It is not an error. There is no `.env` file there, because we never uploaded it. The password comes from the environment variable.
- Find your public link near the top of the service page. It ends in `onrender.com`.

Check: The status is live and you can see the line `Notes Board is running on port`.

If the deploy failed, read the **last red lines** of the log and find the same words in the Stuck Sheet.

Part 7: Test the live site (10 minutes)

- Click your `onrender.com` link. It opens in a new tab.
- Look at the address bar. It must start with `https://`, and you should see a padlock.
- The first visit may be slow. This is normal on the Free plan (see Part 8).
- You see your Notes Board with 3 starter notes.
- Add a note. Write a title, a text and the password you set in Part 5.

Check: The note appears and the message says **Note added**.

- Try a **wrong password**. You see **Wrong password**. This is the server saying 401 (not allowed).
- Delete your note with the right password.

Check: The note disappears.

- Open the `/health` path. Add `/health` to the end of your link, for example `https://yourname-notes-board.onrender.com/health`.

Check: You see `{"ok":true}`.

- Open the About page of your live site.

Write your public link here: _____

This is **Tier 3**.

Part 8: Break it on purpose: see the data reset (10 minutes)

On the Free plan, the filesystem is **ephemeral**. Render's documentation says local SQLite data is lost on a redeploy, a restart or a spin down. Let us see it.

- On your live site, add **two notes** with titles `Test A` and `Test B`. Make sure they show on the page.

2. Go to your service page on the Render dashboard.
3. Restart the service. Look for **Manual Deploy** or **Restart service** (the menu names are **check against your dashboard, not verified**). Choose **Restart** if you see it. Otherwise choose **Manual Deploy**, then **Deploy latest commit**.
4. Wait until the service is live again. Watch the log.
5. Refresh your live site.

Check: Test A and Test B are **gone**. The 3 starter notes (Welcome , Study plan , Idea) are back.

Why? When the service starts again, the old `notes.sqlite` file is gone. Our `data.js` finds an empty table and adds the 3 starter notes again.

Write two sentences in your notebook:

1. What happened to my notes: _____
2. What a real app would need to keep the notes: _____

(Answer: a hosted database that is separate from the app, or a paid persistent disk. Render's documentation says persistent disks are not available on the Free plan, and that a free Render PostgreSQL database expires 30 days after it is created.)

Two more facts to remember

- The Free service **spins down** after 15 minutes without visitors. The next visit takes about one minute to start it again. When this happens, the data is also lost.
- There are **750 free instance hours** a month per workspace. Render may also restart a Free service at any time.

Part 9: Update the site and watch the auto-deploy (5 minutes)

Because you used the **Git Provider** route, Render's documentation says that auto-deploys are supported. That means a new push can start a new deploy. **How the first redeploy behaves on your service is not tested.** Let us find out.

1. In VS Code, open `public/index.html`.
2. Find the footer line: `<p>Built in the Web Technology workshop at Mandsaur University.</p>`
3. Change the text a little, for example add your name at the end. Save.
4. Commit and push:

```
git add .
```

```
git commit -m "Change the footer text"
```

```
git push
```

5. Go to the **Deploys** page of your service on Render.

Check: Does a new deploy start by itself? Write what you saw: _____

If no deploy starts after a few minutes, use **Manual Deploy**. Tell your instructor what you saw.

6. When it is live, refresh your site. The new footer text should show.

If time is short, your instructor may do this part as a demo only.

Part 10: Wrap up (15 minutes)

10a. The showcase (in pods)

Sit in your pod (a group of about six students). One at a time, each student shows the work:

1. Show your **live link** (Tier 3) or your **localhost screen** (Tier 1 or 2).
2. Add a note and delete a note.
3. Say in one sentence: "What I built."
4. Say in one sentence: "What I would fix next."

When others open your live link, they can read your notes. Do **not** tell them your password unless you want them to add notes.

10b. Instructor checklist for the showcase (the instructor ticks this)

Works on localhost (Tier 1)

Pushed to GitHub, with no `.env` file in the repository (Tier 2)

Live on Render, with an `https://` link (Tier 3)

The student can explain why the notes were reset

The student can explain where the password is kept

10c. Quiz and feedback

1. Take the Day 5 quiz.
2. Fill in the feedback. Your instructor will tell you how. Three questions: What was the best part? What was the hardest part? What should we change?

10d. Your completion checklist

This is a checklist for you. It is not an official certificate.

- I installed Node.js and Git, and I used the terminal
- I built a frontend with HTML, CSS and JavaScript
- I built a backend with Node.js and Express
- I saved data in a SQLite database
- I used Git and pushed my code to GitHub
- I kept my password out of my code and out of GitHub
- I deployed to Render, or I understand how (circle one)
- I can explain why the data reset on the Free plan

Name: _____ Date: _____

Day 5 extras (for fast finishers)

These use only what you already know. Always test on localhost first, then commit and push.

- Edit the About page.** Open `public/js/student-details.js`. Change your details. Save, test, commit and push. Watch the Deploys page.
- A "few characters left" message (this extra was not tested).** The page already shows `0 / 300 characters`. Add a warning when fewer than 20 characters are left. In `public/js/app.js`, find section 8 and make it look like this:

File: `public/js/app.js`

```
// 8. Show how many characters are used while the user types.
textInput.addEventListener("input", function () {
  charCount.textContent = textInput.value.length;

  const left = 300 - textInput.value.length;
  if (left < 20) {
    showMessage("Only " + left + " characters left.", true);
  } else {
    showMessage("", false);
  }
});
```

- Count wording.** The page already shows the number of notes in a small badge. Change the empty-state text "No notes yet. Add your first note above." in `renderNotes` to your own words.

4. **Change the starter notes.** In `data.js`, change the text of the 3 starter notes. To see the new text on localhost, stop the server, delete the file `notes.sqlite` from your `notes-board` folder, and start the server again. This is the same reset that Render does to you.
 5. **Improve your README.** Add a line about what you learned, and a line with your live link.
-

I am far behind

This is okay.

1. Copy the **done** folder for Day 5 from your instructor and compare it with your own work.
2. Make sure you reach **Tier 1** (works on localhost). That is the pass.
3. If you can, reach **Tier 2** (pushed to GitHub). Then try Render at home, with your instructor's lab sheet.